

Functioneel ontwerp — ScreenFind

Leerlijn Frontend — Eindopdracht V3.6

Student	[vul naam in]
Studentnummer	[vul in]
Inleverdatum	[vul in]
Docent	[vul in]

Inhoudsopgave

1. [Inleiding](#)
2. [Algemene omschrijving applicatie](#)
3. [Use case tabellen](#)
4. [Functionele en niet-functionele eisen](#)
5. [Inspiratiebronnen](#)
6. [Wireframes](#)
7. [Schermontwerpen](#)
8. [Bronnenlijst](#)
9. [Bijlage A — AI-prompts](#)

1. Inleiding

Dit functioneel ontwerp beschrijft ScreenFind: een React-frontend waarmee gebruikers films, series en personen kunnen zoeken via TMDB, details kunnen bekijken en — na authenticatie via de NOVI Dynamic API — een persoonlijke watchlist kunnen beheren. Het document bevat user stories, use cases, eisen, inspiratie, wireframes en schermontwerpen, zodat een ontwikkelaar zonder verdere ontwerp vragen kan bouwen.

2. Algemene omschrijving applicatie

Probleem

Kijkers verspreiden “nog kijken”-lijstjes over meerdere apps en notities. Zoeken, details raadplegen en bijhouden wat je wilt zien gebeurt daardoor versnipperd.

Oplossing

ScreenFind bundelt zoeken (TMDB), detailpagina's en een afgeschermd watchlist (NOVI) in één applicatie met duidelijke navigatie en responsive layout.

User stories (vier kernfunctionaliteiten)

1. **Als** bezoeker **wil ik** me registreren en inloggen **zodat** ik mijn watchlist veilig kan opslaan en later opnieuw kan openen.
2. **Als** bezoeker **wil ik** films, series en personen kunnen zoeken en filteren op mediatype **zodat** ik snel relevante resultaten vind.

3. **Als** ingelogde gebruiker **wil ik** items aan mijn watchlist toevoegen, de status wijzigen en items verwijderen **zodat** ik overzicht houd over wat ik nog wil kijken.
 4. **Als** gebruiker **wil ik** een detailpagina van een film, serie of persoon zien (met watchlist-actie indien ingelogd) **zodat** ik een weloverwogen kijkkeuze kan maken.
-

3. Use case tabellen

UC-01 — Inloggen

Veld	Inhoud
Use case naam	Inloggen
Actor	Bezoeker / geregistreerde gebruiker
Preconditie	Gebruiker heeft een account; loginpagina is bereikbaar
Trigger	Gebruiker opent Login en dient het formulier in
Main success scenario	1. Gebruiker navigeert naar <code>/login</code> . 2. Gebruiker vult e-mail en wachtwoord in. 3. Systeem stuurt credentials naar NOVI (<code>POST /api/login</code>). 4. Systeem ontvangt JWT, slaat token op, zet Auth Context. 5. Gebruiker wordt doorgestuurd naar <code>/watchlist</code> .
Alternatief 1	Ongeldige credentials → foutmelding in UI, gebruiker blijft op loginpagina.
Alternatief 2	Netwerk-/serverfout → generieke foutmelding + mogelijkheid opnieuw te proberen.
Alternatief 3	Lege velden → client-side validatie voorkomt submit.
Postconditie	Gebruiker is geauthenticeerd; private routes zijn toegankelijk.

UC-02 — Zoeken en filteren

Veld	Inhoud
Use case naam	Zoeken en filteren van media
Actor	Bezoeker of ingelogde gebruiker
Preconditie	Search-pagina is bereikbaar; TMDB-credentials zijn geconfigureerd
Trigger	Gebruiker dient een zoekopdracht in
Main success scenario	1. Gebruiker opent <code>/search</code> . 2. Gebruiker typt query en bevestigt. 3. Systeem toont laadstatus. 4. Systeem haalt resultaten op via TMDB <code>/search/multi</code> . 5. Resultaten verschijnen als kaarten. 6. Gebruiker filtert optioneel op movie/tv/person.
Alternatief 1	Geen resultaten → empty state met duidelijke tekst.
Alternatief 2	API-fout → foutmelding in UI.
Alternatief 3	Lege query → geen request; focus blijft op input.
Postconditie	Resultaten of statusmelding zijn zichtbaar.

UC-03 — Watchlist beheren

Veld	Inhoud
Use case naam	Watchlist beheren
Actor	Ingelogde gebruiker
Preconditie	Geldige JWT; gebruiker heeft toegang tot <code>/watchlist</code>
Trigger	Gebruiker voegt toe, wijzigt status of verwijdert een item
Main success scenario	1. Gebruiker opent watchlist of detailpagina. 2. Gebruiker voegt item toe (POST) of wijzigt status (PATCH) of verwijdert (DELETE). 3. Systeem toont laadfeedback. 4. Lijst/UI wordt bijgewerkt.
Alternatief 1	Niet ingelogd → redirect naar <code>/login</code> .
Alternatief 2	Item bestaat al → informatieve melding, geen duplicaat.
Alternatief 3	API-fout → foutmelding; bestaande lijst blijft zichtbaar.
Postconditie	Watchlist weerspiegelt de laatste succesvolle wijziging.

UC-04 — Detailpagina bekijken

Veld	Inhoud
Use case naam	Detailpagina bekijken
Actor	Bezoeker of ingelogde gebruiker
Preconditie	Geldig <code>mediaType</code> + <code>id</code> in de URL
Trigger	Gebruiker klikt een resultaatkaart of opent een deep link
Main success scenario	1. Gebruiker navigeert naar <code>/details/:mediaType/:id</code> . 2. Systeem toont laadstatus. 3. Systeem haalt detailgegevens op bij TMDB. 4. Pagina toont titel, poster, synopsis/metadata. 5. Indien ingelogd: knop om aan watchlist toe te voegen.
Alternatief 1	Onbekend <code>id</code> → fout-/niet-gevonden melding.
Alternatief 2	API-fout → foutmelding met terug-link naar search.
Alternatief 3	Niet-ondersteund <code>mediatype</code> → 404/Not found.
Postconditie	Detailinformatie of foutstatus is zichtbaar.

4. Functionele en niet-functionele eisen

Functionele eisen

1. Als bezoeker kan ik de homepage met hero-animatie bekijken.
2. Als bezoeker kan ik via het menu naar Home, Search, Contact, Sign-in en Watchlist navigeren.
3. Als bezoeker kan ik me registreren met e-mail, wachtwoord en optionele gebruikersnaam.
4. Als bezoeker kan ik inloggen met e-mail en wachtwoord.

5. Als gebruiker kan ik uitloggen, waarna mijn sessie wordt beëindigd.
6. Als bezoeker kan ik films, series en personen zoeken via TMDB.
7. Als bezoeker kan ik zoekresultaten filteren op mediatype (movie, tv, person).
8. Als bezoeker zie ik een laadindicatie tijdens API-requests.
9. Als bezoeker zie ik een foutmelding wanneer een API-request mislukt.
10. Als bezoeker zie ik een empty state wanneer er geen zoekresultaten zijn.
11. Als bezoeker kan ik vanuit een resultaatkaart naar een detailpagina navigeren.
12. Als bezoeker kan ik op de detailpagina synopsis en metadata bekijken.
13. Als ingelogde gebruiker kan ik een film of serie aan mijn watchlist toevoegen.
14. Als ingelogde gebruiker kan ik mijn watchlist bekijken op een private route.
15. Als ingelogde gebruiker kan ik de status van een watchlist-item wijzigen (to-watch / watching / watched).
16. Als ingelogde gebruiker kan ik een item uit mijn watchlist verwijderen.
17. Als niet-ingelogde gebruiker word ik bij `/watchlist` doorgestuurd naar login.
18. Als bezoeker kan ik een contactsectie/pagina openen.
19. Als bezoeker kan ik de applicatie bedienen via React Router-routes.
20. Als ontwikkelaar lever ik API-keys via een `.env` -bestand aan.

Niet-functionele eisen

21. Als gebruiker wil ik dat de layout responsive is op desktop en smaller viewport.
22. Als gebruiker wil ik dat styling handgeschreven CSS + Flexbox is (geen Bootstrap/MUI/Tailwind).
23. Als gebruiker wil ik dat authenticatiestatus via React Context beschikbaar is.
24. Als ontwikkelaar wil ik dat de code in JavaScript (geen TypeScript) is geschreven.
25. Als ontwikkelaar wil ik dat de applicatie zonder crash opstart (`npm run dev`).
26. Als gebruiker wil ik dat private content niet zonder geldige token zichtbaar is.
27. Als ontwikkelaar wil ik semantische HTML met passende attributen (`aria-*` , labels).
28. Als ontwikkelaar wil ik dat broncode via Git met kleine commits en pull requests wordt beheerd.

5. Inspiratiebronnen

Voeg in de PDF-export screenshots toe van de onderstaande bronnen.

1. Letterboxd

- **Waarom:** heldere postergrids, snelle scanbaarheid van titels.
- **Wat overgenomen:** resultaatkaarten met poster + titel + subtitel; donkere sfeer.

2. TMDB website

- **Waarom:** bekende film-/serie-detailopbouw (poster links, tekst rechts).
- **Wat overgenomen:** detailpagina-layout en badge voor mediatype.

3. Netflix / streaming search UI

- **Waarom:** gecentreerde search-ervaring en focus op één primaire actie.
- **Wat overgenomen:** Google-achtige lege search-state die verschuift zodra er resultaten zijn.

6. Wireframes

Papier-wireframes (Quickscan-eis): teken de vijf pagina's hieronder op papier, fotografeer/scan ze leesbaar, en voeg ze hier in. Printbare SVG-sjablonen staan in `docs/wireframes/` .

#	Pagina	Bestand (sjabloon)
---	--------	--------------------

1	Home	docs/wireframes/01-home.svg
2	Search	docs/wireframes/02-search.svg
3	Login / Register	docs/wireframes/03-auth.svg
4	Detail	docs/wireframes/04-detail.svg
5	Watchlist	docs/wireframes/05-watchlist.svg

Bij elke foto: paginatitel + korte beschrijving van de zones (nav, content, CTA's).

7. Schermontwerpen

Maak in **Figma** (of Adobe XD) minimaal vijf desktop schermen die 1-op-1 overeenkomen met de wireframes, met genoeg detail (kleuren, typografie Nunito, spacing, componentstates) zodat tijdens het bouwen geen ontwerpkeuzes meer nodig zijn.

Vereiste schermen:

1. Home (hero + typewriter)
2. Search (leeg + met resultaten)
3. Login
4. Detail (film/serie)
5. Watchlist (ingelogd)

Figma-link (openbaar): [plak hier de publieke Figma-link]

Exporteer screenshots met titel + beschrijving in deze PDF.

8. Bronnenlijst

NOVI Hogeschool. (z.d.). *Eindopdracht leerlijn Frontend (V3.6)*.

The Movie Database. (z.d.). *API documentation*. <https://developer.themoviedb.org/docs>

NOVI. (z.d.). *NOVI Dynamic API documentation*. <https://novi-backend-api-wgsgz.ondigitalocean.app/documentation/1-Overview>

Cursor. (2026). *AI coding assistant* [Large language model]. <https://cursor.com>

Bijlage A — AI-prompts

Noteer hier de prompts die je met Cursor/ChatGPT hebt gebruikt, bijvoorbeeld:

1. "Extract all requirements from Eindopdracht Frontend V3.6 PDF..."
2. "Implement React Router, Auth Context and NOVI login for ScreenFind..."
3. (vul aan tijdens het project)